JobLens Viva Guide

Detailed and easy-to-learn explanation of the JobLens project for viva, demo, and interview preparation.

> **One-line explanation:** JobLens is a full-stack job application tracker where users can register, log in securely, add job applications, update their status, and view dashboard analytics.

Frontend: HTML + CSS + JavaScript **Backend: Node.js + Express** **Database: MongoDB + Mongoose** **Security: bcrypt + JWT**

1. Project Overview

The project helps a user track job applications. A user can create an account, sign in, add job applications, edit them, delete them, filter them, and see analytics like total applications, interview count, offers, and charts.

Main features

- User authentication with password hashing and JWT.
- Create, read, update, and delete job applications.
- Dashboard stats and analytics charts.
- Demo mode if backend is not available.
- Responsive UI with sidebar, modal, cards, and filters.

Folder structure

Path	Purpose
`frontend/index.html`	Main UI structure. Contains login page, register page, dashboard, jobs page, analytics page, modal, and toast container.
`frontend/style.css`	All styling for layout, cards, sidebar, modal, charts, buttons, responsive design, and colors.
`frontend/app.js`	Main frontend logic. Handles authentication flow, page switching, rendering, filtering, CRUD calls, demo mode, modal logic, and charts.
`backend/server.js`	Express server, MongoDB connection, Mongoose schemas, authentication middleware, and all API routes.
`backend/package.json`	Lists backend dependencies and scripts.

2. End-to-End Working Flow

1. User opens the app in the browser.
2. Frontend loads `index.html`, styles from `style.css`, and behavior from `app.js`.
3. If user already has token and user data in `localStorage`, the app enters the main application.
4. If user logs in or registers, frontend sends request to backend API.
5. Backend validates request, accesses MongoDB, then returns JSON data.
6. Frontend stores token and user info in `localStorage`.
7. Frontend fetches job data and renders dashboard, jobs list, and analytics.
8. Every protected API request sends `Authorization: Bearer token`.

> **Important viva sentence:** The frontend is responsible for UI and user interaction, while the backend is responsible for authentication, validation, database access, and secured CRUD operations.

3. Frontend: index.html Explained

`index.html` is the skeleton of the app. It contains all visible sections, and JavaScript shows or hides them as needed.

Important sections in index.html

Section	What it does
Mobile menu button	Calls `toggleSidebar()` so sidebar can open on small screens.
`#authWrap`	Wrapper for authentication area. Contains login and register screens.
`#loginPage`	Login form with email, password, and Sign In button.
`#registerPage`	Registration form with name, email, password, and Create Account button.
`#appWrap`	Main application wrapper shown after login.
Sidebar	Contains navigation items: Dashboard, Applications, Analytics, plus sign out button.
`#dashPage`	Dashboard page with overview stats, funnel, and recent applications.
`#jobsPage`	Applications page with search, filters, sort controls, and all job cards.
`#analyticsPage`	Analytics page with summary stats, 3 charts, and recent activity list.
`#jobModal`	Modal used for both adding and editing job applications.
`#toastContainer`	Holds toast messages like success, error, or info alerts.

Why inline onclick is used here

This project uses inline event handlers like `onclick="doLogin()"`. This is easy for beginners to understand because the HTML directly shows which function runs on click. In bigger production apps, developers usually attach events from JavaScript instead.

4. Frontend: style.css Explained

`style.css` controls the visual design. It does not change data or logic. It only decides how the app looks.

Important CSS parts

Section	Purpose
`:root` variables	Stores theme colors, spacing, border radius, shadows, and reusable design tokens.
Layout classes	Create app structure with sidebar on left and main content on right.
Auth styles	Design login and register cards.
Button styles	Reusable button system: normal, ghost, danger, small buttons.
Job card styles	Styles each application card and expanded detail section.
Modal styles	Controls overlay, card box, scroll behavior, and close button.
Toast styles	Displays success and error notifications in the bottom-right corner.
Responsive media query	Hides sidebar on mobile and shows menu button.

What to say in viva about CSS

- The UI uses CSS custom properties for consistent theming.
- The layout uses flexbox and grid for responsive structure.
- Separate classes are used for card states, button variants, and status badges.
- The app is mobile-friendly because of the media query for screen widths below 768px.

5. Frontend JavaScript: app.js Complete Explanation

`app.js` is the brain of the frontend. It stores data, talks to backend APIs, reacts to button clicks, updates the UI, and builds charts.

5.1 Global constants and state

Name	Meaning
`API`	Base URL of backend server, currently `http://localhost:5000`.
`STATUS_OPTIONS`	List of valid statuses shown in dropdowns.
`PRIORITY_OPTIONS`	List of valid priorities shown in dropdowns.
`allJobs`	Stores all job data currently loaded on frontend.
`editingJobId`	Stores which job is currently being edited in the modal.
`chartInstances`	Stores Chart.js chart objects so old charts can be destroyed before redrawing.

5.2 Utility functions

`toast(msg, type)`

- **Purpose:** Shows a temporary message to user.
- **Input:** Message text and type such as success, error, or info.
- **How it works:** Creates a new div, adds it to toast container, then removes it after a timeout.
- **Why important:** Gives user feedback after actions like save, delete, or login failure.

`getToken()`

- Reads JWT token from `localStorage`.
- Used whenever frontend needs to call protected APIs.

`getUser()`

- Reads logged-in user data from `localStorage`.
- Used to show name, email, avatar initials, and greeting.

`authHeaders()`

- Builds request headers for authenticated API calls.
- Sends JSON content type and bearer token.

`escHtml(str)` and `escAttr(str)`

- Escape special characters before inserting user data into HTML.
- These functions help prevent broken markup and reduce XSS risk.

`encodeInlineJob(job)`

- Converts a job object into a safe string so it can be used inside inline HTML button handlers.
- This is used by the Edit button.

5.3 Navigation and page switching

`showPage(id)`

- Used in authentication area.
- Hides login and register sections, then shows the selected page.
- Example: switching from login to register page.

`switchPage(id, el)`

- Used in main application area.
- Changes active page between Dashboard, Applications, and Analytics.
- Also updates active sidebar item.
- If Analytics page is opened, it calls `renderAnalytics()`.
- If Jobs page is opened, it calls `filterJobs()`.
- If Dashboard page is opened, it calls `renderDash()`.

`toggleSidebar()`

- Opens or closes sidebar on smaller screens.
- Used by the mobile menu button.

5.4 Authentication functions

`doLogin()`

- Reads email and password from form.
- Validates that fields are not empty.
- Sends `POST /login` request to backend.
- If login succeeds, saves token and user data in `localStorage`, then calls `initApp()`.
- If backend is unreachable, it starts demo mode.

Viva point: Demo mode is a fallback so the UI can still be demonstrated even if backend server or database is not running.

`doRegister()`

- Reads full name, email, and password.
- Checks all fields are filled and password length is at least 6.
- Sends `POST /register` request.
- If successful, saves token and user then opens main app.

`doLogout()`

- Clears token and user from `localStorage`.
- Clears frontend job state.
- Shows login page again.

`demoMode()`

- Creates fake demo user and sample job data directly in frontend.
- Used when backend is unavailable.
- Allows viva or demo presentation without database dependency.

5.5 App initialization

`initApp(isDemo = false)`

- Switches UI from auth pages to main app.
- Reads user from storage and displays initials, name, email, and greeting.
- If app is in demo mode, it directly renders dashboard and jobs.
- Otherwise, it loads jobs from backend using `loadJobs()`.

`window.onload`

- Runs automatically when page fully loads.
- If token and user already exist, it restores the session.
- Adds modal close behavior for overlay click.
- Adds keyboard shortcuts:
    - `N` opens Add Application modal.
    - `Escape` closes modal.

5.6 Job API functions

`loadJobs()`

- Calls backend `GET /jobs`.

- Stores response in `allJobs`.
- Then renders dashboard and job list.

`saveJob(data)`

- Handles both create and update logic.
- If `editingJobId` exists, it sends `PUT /jobs/:id`.
- If not, it sends `POST /jobs`.
- In demo mode, it updates local array instead of calling backend.

`deleteJob(id)`

- Asks for confirmation.
- In demo mode, removes item locally.
- In real mode, sends `DELETE /jobs/:id`.

`quickUpdateStatus(id, status)`

- Quickly changes a job's status from dropdown in expanded card.
- Updates frontend and dashboard immediately.
- In live mode, also sends backend update.

`quickUpdate(id, field, value)`

- Generic update function used for simple field updates like priority.
- Useful because the same logic can update different fields without separate functions.

5.7 Modal functions

`openAddModal()`

- Prepares modal for creating a new job.
- Clears all form inputs.
- Sets default status to Applied and priority to Medium.

`openEditModal(job)`

- Prepares modal for editing an existing job.
- Fills form fields using selected job data.
- Stores job id in `editingJobId`.

`closeModal()`

- Removes modal open class so popup disappears.

`submitJob()`

- Collects all form values from modal.
- Validates that company and role are present.
- Calls `saveJob()`.
- On success, closes modal, refreshes UI, and shows toast.

5.8 Rendering functions

`renderDash()`

- Calculates total applications, interview count, offers, rejected jobs, and interview rate.
- Builds stat cards dynamically.
- Builds small funnel view for Applied, Interview, and Offer.
- Shows recent jobs section.

`renderJobs(jobs)`

- Renders job cards list for Applications page.
- Also updates count text such as "8 applications".
- Shows empty state if list is empty.

`jobCardHTML(job)`

- Returns HTML string for a single job card plus its expandable detail section.
- This contains company, role, status, date, quick update dropdowns, Edit button, Open Listing button, and Remove button.

`toggleJobDetail(id)`

- Expands or collapses details of a job card.
- Also closes any other expanded job card first, so only one stays open at a time.

5.9 Filter and sorting functions

`filterJobs()`

- Reads search text, status filter, priority filter, and sort order from controls.
- Filters data from `allJobs`.

- Then sorts and sends result to `renderJobs()`.

`sortJobs(jobs, sort)`

- Sorts by date or company name.
- Supports both ascending and descending order.

5.10 Analytics functions

`renderAnalytics()`

- Builds summary values like total jobs, this week count, offers, and progress rate.
- Groups jobs by status, priority, and day.
- Calls `buildChart()` three times to create graphs.
- Builds recent activity timeline.

`buildChart(id, type, data, options)`

- Reusable helper for Chart.js.
- Destroys previous chart instance before drawing a new one.
- This prevents duplicate charts and memory issues.

> **Most likely frontend viva question:** Why are render functions separate from API functions?
> Because API functions fetch or update data, while render functions only update the UI. This separation makes code easier to understand and maintain.

6. Backend: server.js Complete Explanation

The backend is built using Express and MongoDB. It exposes REST APIs for authentication and job management.

6.1 Imports and setup

Code part	Purpose
`express`	Creates HTTP server and routes.
`mongoose`	Connects Node.js app to MongoDB and defines schemas.
`cors`	Allows frontend to send requests to backend.
`bcryptjs`	Hashes passwords securely.
`jsonwebtoken`	Creates and verifies JWT tokens.
`path`	Helps serve frontend static files.

Server initialization

- `const app = express()` creates the Express app.
- `JWT_SECRET` is used to sign and verify tokens.
- `app.use(cors({ origin: "*" }))` allows requests from any origin.
- `app.use(express.json())` parses incoming JSON request bodies.
- `app.use(express.static(...))` serves frontend files.

6.2 Database connection

The code connects to MongoDB using `mongoose.connect()`. If no environment variable is set, it uses local database `mongodb://localhost:27017/jobtracker`.

Why Mongoose is used: It provides schemas, validation, models, query methods, and easier MongoDB interaction.

6.3 Mongoose schemas and models

User schema

- `name`: required string
- `email`: required, unique, lowercased, trimmed
- `password`: required hashed password
- `timestamps: true`: automatically adds `createdAt` and `updatedAt`

Job schema

- `userId`: links each job to a specific user
- `company`, `role`: required fields

- `status`: enum with fixed valid values
- `priority`: enum with Low, Medium, High
- Other optional fields: location, salary, url, notes, contactName, contactEmail, deadline, tags
- `dateApplied`: default current date

Important viva point: Enums are used so invalid status or priority values cannot be stored in the database.

6.4 Authentication middleware

`auth(req, res, next)`

- Reads token from `Authorization` header.
- If token is missing, returns 401 Unauthorized.
- If token is present, verifies it using `jwt.verify()`.
- If valid, saves decoded user data in `req.user`.
- Calls `next()` to continue to protected route.

Why it matters: It protects job routes so one user cannot see or edit another user's jobs.

6.5 Route-by-route explanation

Route	Purpose
`GET /`	Simple health check route. Returns basic JSON saying server is running.
`POST /register`	Creates a new user account after validation and password hashing.
`POST /login`	Checks email and password, then returns JWT token.
`GET /jobs`	Returns current user's jobs. Supports filters and search.
`POST /jobs`	Creates a new job application for logged-in user.
`PUT /jobs/:id`	Updates one existing job if it belongs to current user.
`DELETE /jobs/:id`	Deletes one job if it belongs to current user.
`GET /stats`	Returns aggregate statistics for current user's jobs.

`POST /register` detailed logic

1. Read `name`, `email`, and `password` from request body.
2. Validate all fields are present.

3. Validate password length.
4. Check if user already exists.
5. Hash password with bcrypt.
6. Create user in MongoDB.
7. Generate JWT token.
8. Return token and safe user info.

`POST /login` detailed logic

1. Read email and password.
2. Find user by email.
3. Compare entered password with hashed password using bcrypt.
4. If valid, generate JWT token.
5. Return token and user data.

`GET /jobs` detailed logic

1. First passes through `auth` middleware.
2. Build filter object using logged-in user's id.
3. Optionally add status, priority, and search conditions.
4. Query MongoDB using `Job.find(filter)`.
5. Sort result and return as JSON.

`POST /jobs` detailed logic

1. Protected route, so token is required.
2. Checks company and role are present.
3. Creates a job document with `userId` from token.
4. Returns newly created job.

`PUT /jobs/:id` detailed logic

1. Protected route.
2. Searches by both job id and user id.
3. This ensures a user can update only their own jobs.
4. `runValidators: true` makes schema validations run during update.

`DELETE /jobs/:id` detailed logic

1. Protected route.
2. Deletes only if job belongs to logged-in user.

3. Returns confirmation JSON.

`GET /stats` detailed logic

1. Reads all jobs of the current user.
2. Counts jobs by status.
3. Counts how many were applied to in the last 30 days.
4. Returns aggregate stats object.

6.6 Server start

The backend runs on port `5000` by default. `app.listen(PORT)` starts the server and begins accepting requests.

7. Security Concepts Used

Concept	How this project uses it
Password hashing	Passwords are hashed with bcrypt before storage. Plain passwords are never stored in database.
JWT authentication	Backend issues token after login/register. Frontend sends that token on protected requests.
Protected routes	`auth` middleware blocks unauthorized access.
Ownership check	Job updates and deletes are filtered by both `_id` and `userId`.
Input validation	Backend checks required fields and schema enums before data is stored.
Frontend escaping	User-provided text is escaped before being injected into HTML.

8. Important Design Decisions

Why localStorage?

It keeps user session active after page refresh. Token and basic user data are stored in browser memory that persists until removed.

Why demo mode?

It allows the application to be shown even when backend or database is not running. That is helpful in viva and portfolio demos.

Why one file for frontend logic?

It keeps the project simple and beginner-friendly, though larger apps should separate modules.

Why Chart.js?

It gives quick and visually clear charts without needing to build chart logic manually.

9. Possible Weak Points You Can Honestly Mention

- The project is easy to understand because logic is centralized, but that also means `app.js` and `server.js` are becoming large.
- CORS is currently open to all origins. In production it should be restricted.
- Frontend uses inline event handlers, which is okay for small projects but less scalable for larger apps.
- Token is stored in localStorage; for stronger production security, HTTP-only cookies are often preferred.
- There are no automated tests yet.

Good viva behavior: If the examiner asks for limitations, do not say "there are no problems." Instead say you deliberately chose a simple architecture for clarity and speed, and mention how you would improve it in a production version.

10. How to Explain This Project in 1 Minute

JobLens is a full-stack job application tracker built using HTML, CSS, and JavaScript on the frontend, and Node.js, Express, and MongoDB on the backend. It allows users to register and log in securely using bcrypt password hashing and JWT-based authentication. After login, users can add, edit, delete, filter, and track job applications. The dashboard shows key statistics, and the analytics section visualizes progress using charts. I also added demo mode so the project can still be shown even if the backend is not running.

11. Common Viva Questions and Ready Answers

Question	Short answer
What problem does your project solve?	It helps users organize and track their job applications and progress.
Why did you use MongoDB?	Because the data is document-based, flexible, and easy to model with Mongoose schemas.
Why use JWT?	JWT allows stateless authentication, so the server can verify user identity on each request without storing session data.
Why use bcrypt?	It securely hashes passwords so plain passwords are never saved.
What is middleware?	Middleware is a function that runs before the final route handler. Here it is used for authentication.
What is CRUD?	Create, Read, Update, Delete. This project supports all four operations on jobs.
How do you ensure one user cannot access another user's jobs?	I verify JWT token and then query jobs by both job id and the logged-in user's id.
What is the role of Mongoose schema?	It defines the structure, types, defaults, and validation rules for MongoDB documents.
How is the dashboard updated?	Frontend stores jobs in `allJobs` and re-renders stats and cards whenever job data changes.
What happens if backend is down?	The app can enter demo mode and use sample data for UI demonstration.

12. Function-by-Function Quick Revision Sheet

Function	Remember it as
`toast`	show message
`getToken`	read token
`getUser`	read user
`authHeaders`	prepare auth request headers
`showPage`	switch login/register screen
`switchPage`	switch main app page
`toggleSidebar`	open mobile sidebar
`doLogin`	login user
`doRegister`	create user account
`doLogout`	clear session
`demoMode`	load sample app data
`initApp`	start main application
`loadJobs`	fetch jobs from backend
`saveJob`	create or update job
`deleteJob`	remove job
`quickUpdateStatus`	change status fast
`quickUpdate`	generic field update
`openAddModal`	prepare add form
`openEditModal`	prepare edit form
`closeModal`	hide popup
`submitJob`	read form and save
`renderDash`	build dashboard
`renderJobs`	build jobs list

`jobCardHTML`	generate one job card
`toggleJobDetail`	expand/collapse details
`filterJobs`	search + filter jobs
`sortJobs`	sort jobs
`renderAnalytics`	build charts and activity list
`buildChart`	draw chart safely
`auth middleware`	protect routes

13. Final Last-Night Revision Notes

- **Remember the stack:** HTML, CSS, JS, Node.js, Express, MongoDB, Mongoose, bcrypt, JWT, Chart.js.
- **Remember the flow:** login/register -> token stored -> authenticated requests -> jobs fetched -> UI rendered.
- **Remember security:** hashed passwords, JWT auth, protected routes, ownership checks.
- **Remember frontend data:** everything is mainly driven by the `allJobs` array.
- **If asked about improvement:** split frontend and backend into modules, restrict CORS, add tests, and use HTTP-only cookies.

Prepared from the current project codebase, including the fixed frontend script version.